



Insección de
Los drones son interesantes.

Resumen: Diseñar un sistema eficiente de enrutamiento de drones que navegue múltiples drones a través de zonas conectadas minimizando giros de simulación y gestionando el movimiento restricciones.

Versión: 1.4

Contenidos

I Prólogo	2
II Instrucciones de IA	3
III Instrucciones Comunes	5
III.1 Reglas Generales	5
III.2 Makefile	5
III.3 Directrices Adicionales	6
IV Introducción	7
V Restricciones	8
VI Deja volar al dron	9
VII Parte Obligatoria	11
VII.1 Requisitos de Búsqueda de Ruta y Algoritmos	11
VII.2 Reglas de Ocupación de Zonas	12
VII.3 Mecánicas de Movimiento y Giro	12
VII.4 Restricciones del Parser	13
VII.5 Formato de Salida de Simulación	14
VII.6 Sistema de Puntuación	14
VII.7 Benchmarks de Rendimiento	16
VIII Requisitos del Readme	19
IX Parte de Bonificación	20
X Envío y revisión por pares	21

Capítulo I

Prólogo

Los drones se han utilizado para guiar ovejas en Nueva Zelanda, reemplazando a los perros pastores por guardianes aéreos zumbantes. En Japón, los edificios de oficinas despliegan drones que reproducen música alta y parpadean luces para literalmente ahuyentar a los empleados sobrecargados hasta sus hogares. Un dron fue entrenado para pintar grafitis en las paredes en pleno vuelo — una mezcla rebelde de tecnología y arte urbano. En Suecia, científicos utilizaron drones para oler las heces de ballena que flotan en el océano para estudiar especies en peligro de extinción. Algunos drones experimentales están diseñados como aves o insectos para espiar sin ser detectados, batiendo alas y todo. Incluso hay un dron que vuela aleteando burbujas de jabón, sin hélices involucradas. En la investigación volcánica, un dron voló directamente hacia una nube de erupción, se derretió en el aire, pero logró enviar datos segundos antes de la desintegración. Y en Corea del Sur, los espectáculos de drones sincronizados han reemplazado a los fuegos artificiales — más seguros, silenciosos y de alguna manera aún más mágicos.

La frase proviene de la idea de que la rueda es una invención brillante que existe desde siempre y funciona muy bien. Dado que no hay nada malo con ella, intentar inventarla de nuevo realmente no ayudaría y podría ser una pérdida de tiempo, especialmente cuando ese tiempo podría dedicarse a resolver nuevos problemas en su lugar.

En programación, esto sucede cuando alguien construye algo desde cero que ya existe, como escribir tu propio algoritmo de ordenamiento o marco cuando versiones sólidas y de código abierto ya existen. Pero no todo es malo: hacerlo tú mismo puede ser una gran forma de aprender cómo funcionan las cosas por dentro. La clave es encontrar un equilibrio: no reconstruir todo, pero tomarse el tiempo para explorar cómo las herramientas que usas realmente funcionan. De esa manera, crecerás como desarrollador sin quedarte atascado reinventando las mismas ruedas de siempre.

Capítulo II

Instrucciones de IA

● Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las variadas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, abórdalas siempre con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formulada o de que el contenido generado sea preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

● Mensaje principal

- 👉 Utiliza IA para reducir tareas repetitivas o tediosas.
- 👉 Desarrolla habilidades de prompting ☐ tanto de codificación como de no codificación ☐ que beneficiarán tu futura carrera.
- 👉 Aprende cómo funcionan los sistemas de IA para anticipar y evitar riesgos, sesgos y problemas éticos comunes.
- 👉 Sigue desarrollando habilidades técnicas y de alto rendimiento trabajando con tus compañeros.
- 👉 Solo utiliza contenido generado por IA que entiendas completamente y por el que puedas asumir la responsabilidad.

● Reglas del aprendiz:

- Deberías tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para poder usarlas de forma ética y reducir sesgos potenciales.
- Deberías reflexionar sobre tu problema antes de prompting - esto te ayuda a escribir prompts más claros, detallados y relevantes usando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar de forma sistemática cualquier cosa generada por IA.
- Siempre busca la revisión de pares ☐ no te fíes únicamente de tu propia validación.

● Resultados de la fase:

- Desarrolla habilidades de prompting tanto de uso general como específicas del dominio.
- Aumenta tu productividad con un uso eficaz de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

● Comentarios y ejemplos:

- Con frecuencia te encontrarás con situaciones □ exámenes, evaluaciones, y más □ en las que debes demostrar una comprensión real. Prepárate, continúa desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con compañeros a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA suelen carecer de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus pares, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden proporcionar perspectivas alternativas y matices valiosos. Confía en ellos como un control de calidad.

✓ Buenas prácticas:

Le pregunto a la IA: “¿Cómo pruebo una función de ordenación?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Mala práctica:

Le pido a la IA que escriba una función completa, la Copio y pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad, y fracaso mi proyecto.

✓ Buenas prácticas:

Uso la IA para ayudar a diseñar un analizador. Luego repaso la lógica con un compañero. Capturamos dos errores y lo reescribimos juntos — mejor, más limpio y totalmente entendido.

✗ Mala práctica:

Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y fracaso mi proyecto.

Capítulo III

Instrucciones comunes

III.1 Reglas generales

- Tu proyecto debe estar escrito en Python 3.10 o posterior.
- Tu proyecto debe adherirse al estándar de codificación flake8.
- Tus funciones deben manejar las excepciones de forma elegante para evitar fallos. Usa bloques try-except para gestionar errores potenciales. Prefiere gestores de contexto para recursos como archivos o conexiones para asegurar una limpieza automática. Si tu programa se bloquea debido a excepciones no manejadas durante la revisión, se considerará no funcional.
- Todos los recursos (p. ej., descriptores de archivos, conexiones de red) deben gestionarse correctamente para evitar pérdidas. Usa gestores de contexto cuando sea posible para un manejo automático.
- Tu código debe incluir indicaciones de tipo para parámetros de funciones, tipos de retorno y variables cuando sea aplicable (usando el módulo typing). Usa mypy para la verificación estática de tipos. Todas las funciones deben pasar mypy sin errores.
- Incluye docstrings en funciones y clases siguiendo PEP 257 (p. ej., estilo Google o NumPy) para documentar el propósito, los parámetros y los retornos.

III.2 Makefile

Incluye un Makefile en tu proyecto para automatizar tareas comunes. Debe contener las reglas siguientes (lint obligatorio implica las banderas especificadas; se recomienda encarecidamente probar `-strict` para verificaciones mejoradas):

- **install:** Instalar dependencias del proyecto utilizando `pip`, `uv`, `pipx`, o cualquier otro gestor de paquetes de tu elección.
- **ejecutar:** Ejecuta el script principal de tu proyecto (p. ej., mediante el intérprete de Python).
- **debug:** Ejecuta el script principal en modo de depuración usando el depurador integrado de Python (p. ej., `pdb`).
- **limpiar:** Eliminar archivos temporales o cachés (p. ej., `__pycache__`, `.mypy_cache`) para mantener limpio el entorno del proyecto.

- **lint:** Ejecuta los comandos `flake8 .` y `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
- **lint-strict (opcional):** Ejecuta los comandos `flake8 .` y `mypy . --strict`

III.3 Directrices adicionales

- Crea programas de prueba para verificar la funcionalidad del proyecto (no presentados ni calificados). Utiliza marcos como `pytest` o `unittest` para pruebas unitarias, cubriendo casos límite.
- Incluye un archivo `.gitignore` para excluir artefactos de Python.
- Se recomienda usar entornos virtuales (p. ej., `venv` o `conda`) para aislar dependencias durante el desarrollo.

Si se aplican requisitos adicionales específicos del proyecto, se indicarán inmediatamente debajo de esta sección.

Capítulo IV

Introducción

Los drones autónomos son el futuro del transporte. Ya se utilizan en muchas industrias, como la agricultura, la construcción y la logística. También se utilizan en operaciones militares, como vigilancia y reconocimiento.

Tu tarea es diseñar un sistema que enrute de manera eficiente una flota de drones desde una base central (inicio) a una ubicación objetivo (fin), mientras navega por esta red dinámica bajo un conjunto de restricciones estrictas y objetivos de optimización.

Se te proporcionará un gráfico que representa la red de zonas, y un conjunto de restricciones que debes respetar.

El gráfico se representa como una red de zonas conectadas, donde las conexiones definen posibles rutas de movimiento entre zonas.

Capítulo V

Restricciones

- Cualquier biblioteca que ayude con la lógica de grafos está prohibida (como networkx, graphlib, etc.).
- El proyecto debe ser completamente tipado. Es obligatorio usar flake8 y mypy.
- El proyecto debe ser completamente orientado a objetos.

Esto tendrá que demostrarse durante la revisión por pares.

Capítulo VI Deja que el dron vuele

Adjunto a este tema, encontrarás múltiples archivos que representan la red de zonas en el siguiente formato:

Ejemplo:

```
nb_drones: 5
start_hub: hub 0 0 [color=green]
end_hub: goal 10 10 [color=yellow]
hub: roof1 3 4 [zone=restricted color=red]
hub: roof2 6 2 [zone=normal color=blue]
hub: corredorA 4 3 [zone=priority color=green max_drones=2]
hub: tunnelB 7 4 [zone=normal color=red]
hub: obstacleX 5 5 [zone=blocked color=gray]
connection: hub-roof1
connection: hub-corredorA
connection: roof1-roof2
connection: roof2-goal
connection: corredorA-tunnelB [max_link_capacity=2]
connection: tunnelB-goal
```

¿Interesante, verdad? Para ser más precisos:

- La primera línea define el número de drones usando nb_drones: <número>.
- Definición de zona en cada línea usando prefijos de tipo:
 - start_hub: <nombre> <x> <y> [metadata] marca la zona de inicio.
 - end_hub: <nombre> <x> <y> [metadata] marca la zona final.
 - hub: <nombre> <x> <y> [metadata] define una zona regular.
 - La sintaxis de conexión prohíbe guiones en los nombres de las zonas (ver abajo).
- Todos los metadatos son opcionales y van entre corchetes [...] con valores predeterminados:
 - zone=<tipo> (predeterminado: normal)
 - color=<valor> (predeterminado: ninguno)
 - max_drones=<número> (predeterminado: 1) - Máximo de drones que pueden ocupar esta zona simultáneamente
 - Las etiquetas dentro de los corchetes pueden aparecer en cualquier orden.
- Tipos de zona:

- normal - Zona estándar con 1 turno de coste de movimiento (predeterminado)
- bloqueado - Zona inaccesible. Los drones no deben entrar ni pasar por esta zona. Cualquier ruta que la use es inválida.
- restringido - Una zona sensible o peligrosa. El movimiento a esta zona cuesta 2 turnos.
- prioridad - Una zona preferida. El movimiento a esta zona cuesta 1 turno pero debe priorizarse en la búsqueda de rutas.
- Colores:
 - Los colores son opcionales y se pueden usar para representación visual (salida de terminal o visualización gráfica).
 - Los valores aceptados para el color son cadenas válidas de una sola palabra (p. ej., rojo, azul, grises). No hay una lista fija de colores permitidos.
 - Cuando se especifican colores, la implementación debe proporcionar retroalimentación visual mediante salida de terminal en color o representación gráfica.
- Las conexiones se definen usando connection: <name1>-<name2> [metadata]:
 - Define una conexión bidireccional (arista) entre dos zonas.
 - La sintaxis de conexión prohíbe guiones en los nombres de las zonas.
 - Los metadatos opcionales pueden especificarse entre corchetes [...]:
 - * max_link_capacity=<number> (predeterminado: 1) - Máximo de drones que pueden atravesar esta conexión simultáneamente
- Los comentarios comienzan con # y se ignoran.



Las coordenadas de las zonas siempre serán enteros, y siempre habrá una zona de inicio y una zona final únicas.

Capítulo VII

Parte Obligatoria

Como ya habrás adivinado, el objetivo principal es mover todos los drones desde la zona de inicio hasta la zona final en la menor cantidad de turnos de simulación posible.

VII.1 Requisitos de Búsqueda de Ruta y Algoritmo

- Los drones pueden moverse simultáneamente. El algoritmo debe programar rutas para maximizar el rendimiento y evitar demoras innecesarias.
- Tu implementación debe manejar:
 - Distribución de drones a través de múltiples rutas.
 - Esperas estratégicas cuando el movimiento no es posible.
 - Prevención de conflictos de ruta y bloqueos.
- El algoritmo debe tener en cuenta:
 - Longitudes de ruta, incluyendo costos de movimiento asociados con tipos de zona (p. ej., restringido o prioridad).
 - Programación de turnos, para evitar colisiones o bloqueo entre drones.
 - Estructura del grafo, para determinar rutas disponibles disjuntas o que se superponen.
 - Restricciones de capacidad de zona (`max_drones`) y capacidad de conexión (`max_link_capacity`).
- Su algoritmo debe ser adaptable: diferentes mapas pueden requerir estrategias de enrutamiento distintas, según la topología y los tipos de zonas.
- **Representación Visual: la** Tu implementación debe proporcionar retroalimentación visual de simulación, ya sea mediante:
 - Salida en terminal coloreada que muestre movimientos de drones y estados de las zonas
 - Una interfaz gráfica que muestre la red y las posiciones de los drones
 - Ambas opciones para una experiencia de usuario mejorada



- ¿Qué tan eficiente es tu algoritmo?
- ¿Puede funcionar con un gran número de drones?
- ¿Cuál es la complejidad (p. ej., $O(n)$, $O(\log n)$, etc.)?
- ¿Estás recalculando o almacenando en caché rutas?
- ¿Cómo afecta el uso de memoria?
- ¿Cómo mejora tu representación visual la comprensión de la simulación?

VII.2 Reglas de Ocupación de Zonas

- Por defecto, una zona puede contener como máximo un dron en cada turno de simulación.
- Las zonas con metadatos `max_drones=N` pueden contener hasta N drones simultáneamente.
- Las únicas excepciones especiales a las reglas de ocupación son:
 - La zona de inicio: todos los drones comienzan aquí y pueden compartir el espacio inicialmente.
 - La zona final: varios drones pueden llegar aquí y se consideran entregados.
- Dos drones no pueden entrar en la misma zona en el mismo turno a menos que la capacidad de la zona lo permita.
- Un dron no puede moverse a una zona que exceda su capacidad máxima.
- La capacidad de conexión (`max_link_capacity`) definida en las conexiones limita cuántos drones pueden atravesar la misma conexión simultáneamente.
- Los drones pueden moverse simultáneamente, siempre que se respeten todas las restricciones de capacidad.

VII.3 Mecánica de Movimiento y Turnos

La simulación procede en turnos discretos. En cada turno, cada dron puede:

- Moverse a una zona conectada adyacente (si la capacidad lo permite).
- Moverse a una conexión hacia una zona restringida (que requiere 2 turnos para alcanzarse). En este caso, el dron DEBE llegar a su destino durante el siguiente turno. No puede esperar turnos extra en la conexión.
- Quedarse en el lugar (p. ej., para esperar, o si el movimiento está bloqueado).

La simulación debe evitar conflictos y asegurar una programación de movimientos válida basada en la evaluación del estado turno a turno:

- Los drones que salen de una zona liberan capacidad para ese mismo turno.
- Una zona debe tener capacidad disponible para que un dron entre en ella (después de que todos los drones que salen liberen espacio).

- Para movimientos de múltiples turnos (zonas restringidas), el dron ocupa la conexión durante el tránsito y DEBE llegar al destino después del número especificado de turnos. No puede esperar en la conexión a un espacio vacío en la zona de destino.

Cada movimiento entre zonas tiene un costo en turnos, basado en el tipo de zona de destino:

- normal: 1 turno (predeterminado)
- restringido: 2 turnos
- prioridad: 1 turno (pero debe preferirse en algoritmos de búsqueda de ruta)
- bloqueado: Inaccesible – no puede ser entrado

VII.4 Restricciones del analizador

El archivo de entrada debe respetar la estructura y sintaxis esperadas:

- La primera línea debe definir el número de drones usando nb_drones: <entero_positivo>.
- El programa debe poder manejar cualquier cantidad de drones.
- Debe haber exactamente un start_hub: zona y un end_hub: zona.
- Cada zona debe tener un nombre único y coordenadas enteras válidas.
- Los nombres de zona pueden usar cualquier carácter válido excepto guiones y espacios.
- Las conexiones deben enlazar solo zonas definidas previamente usando connection: <zona1>-<zona2> [metadata].
- La misma conexión no debe aparecer más de una vez (p. ej., a-b y b-a se consideran duplicados).
- Cualquier bloque de metadatos (p. ej., [zone=... color=...] para zonas, [max_link_capacity=...] para conexiones) debe ser sintácticamente válido.
- Los tipos de zona deben ser uno de: normal, blocked, restricted, priority. Cualquier tipo inválido debe generar un error de análisis.
- Los valores de capacidad (max_drones para zonas, max_link_capacity para conexiones) deben ser enteros positivos.
- C cualquier otro error de análisis debe detener el programa y devolver un mensaje de error claro que indique la línea y la causa.



Se recomienda encarecidamente crear tus propios archivos de mapa además de los proporcionados en la materia para manejar casos límite y errores.

VII.5 Formato de Salida de la Simulación

- La simulación debe generar el movimiento paso a paso de los drones desde la zona de inicio hasta la zona final.
- Cada turno de simulación se representa en una línea.
- Una línea debe enumerar todos los movimientos de drones que ocurren durante ese turno, separados por espacios. Cada movimiento debe seguir el formato: D<ID>-<zona>, o D<ID>-<conexión> en el caso de drones aún en vuelo hacia zonas restringidas.
 - D<ID> se refiere al identificador único del dron (por ejemplo, D1, D2).
 - <zona> es el nombre de la zona de destino.
 - <conexión> es el nombre de la conexión hacia una zona restringida.
- Los drones que no se mueven en un turno dado se omiten en esa línea.
- Los drones que alcanzan la zona final se consideran entregados y ya no se rastrean.
- La simulación termina cuando todos los drones han llegado a la zona final.
- Ejemplo:

D1-techo1 D2-corridorA
D1-techo2 D2-túnelB
D1-meta D2-meta

VII.6 Sistema de Puntuación

- El rendimiento de una solución se evalúa en función del número total de turnos de simulación necesarios para enrutar todos los drones desde la zona de inicio hasta la zona final.
- Cuantos menos turnos, mejor puntuación.
- Una simulación válida debe:
 - Cumplir con todas las reglas de movimiento y ocupación.
 - Manejar correctamente los costos de movimiento asociados con los tipos de zona.
 - Respetar todas las limitaciones de capacidad (límites de zonas y conexiones).
 - Evitar todos los conflictos (p. ej., exceder la capacidad de zona o conexión).

Métricas de evaluación secundarias (opcionales) pueden incluir:

- El número de drones movidos por turno (eficiencia de la asignación de rutas).
- El número medio de turnos por dron.
- El costo total del camino (suma de costos de movimiento ponderados entre todos los drones).

- Calidad y utilidad de la representación visual.

En caso de conteos de giros idénticos, las soluciones pueden compararse en función de métricas secundarias o la calidad del código.



Estas métricas secundarias no son obligatorias para calcularse automáticamente, pero se anima a los estudiantes a mostrarlas en su salida de simulación o documentación para ayudar a otros a evaluar el rendimiento.

VII.7 Benchmarks de rendimiento

Los siguientes objetivos de rendimiento definen el nivel de optimización esperado que debe lograr su implementación.

• Rendimiento esperado:

- Los mapas fáciles deben resolverse en menos de 10 turnos
- Los mapas medios deben resolverse en 10530 turnos
- Los mapas difíciles deben resolverse en menos de 60 turnos
- El mapa Challenger (opcional) debe intentar superar el récord de referencia de 45 turnos Este nivel es puramente opcional y no afecta su calificación.

Para ayudar a evaluar la eficiencia de su algoritmo, aquí hay objetivos de rendimiento de referencia basados en los mapas de prueba proporcionados:

• Mapas Fáciles:

- Camino lineal con 2 drones: Objetivo ^{INIA} 6 turnos
- bifurcación simple con 4 drones: Objetivo 8 turnos
- Capacidad básica con 4 drones: Objetivo ^{IN} 6 turnos

• Mapas Medios:

- Senda sin salida con 5 drones: Objetivo ^{IN} 12 turnos
- Bucle circular con 6 drones: Objetivo ^N 15 turnos
- Puzzle prioritario con 5 drones: Objetivo ^{IA} 12 turnos

• Mapas Difíciles:

- Pesadilla de laberinto con 8 drones: Objetivo ^{IN} 30 turnos
- Infierno de capacidad con 12 drones: Objetivo ^{IN} 35 turnos
- Desafío definitivo con 15 drones: Objetivo ^{IN} 45 turnos

• Mapa Challenger (opcional □ para implementaciones excepcionales):

- El Sueño Imposible con 25 drones: Récord de referencia: 45 turnos
- Este desafío casi irresoluble está diseñado para investigación y optimización algorítmica
- Resolver este mapa demuestra habilidades excepcionales de búsqueda de rutas y optimización
- Nota: Este nivel es puramente opcional y no afecta su calificación



Estos benchmarks se proporcionan como objetivos de optimización para ayudar a evaluar el rendimiento de su algoritmo. Alcanzar estos objetivos demuestra una implementación bien optimizada y será evaluada durante la evaluación entre pares.



- ¿Puede tu algoritmo cumplir con estos puntos de referencia de rendimiento?
- ¿Cómo se compara tu solución con los objetivos de referencia?
- ¿Qué optimizaciones implementaste para lograr un mejor rendimiento?
- ¿Puedes resolver el mapa Challenger y batir el récord de 45 giros?

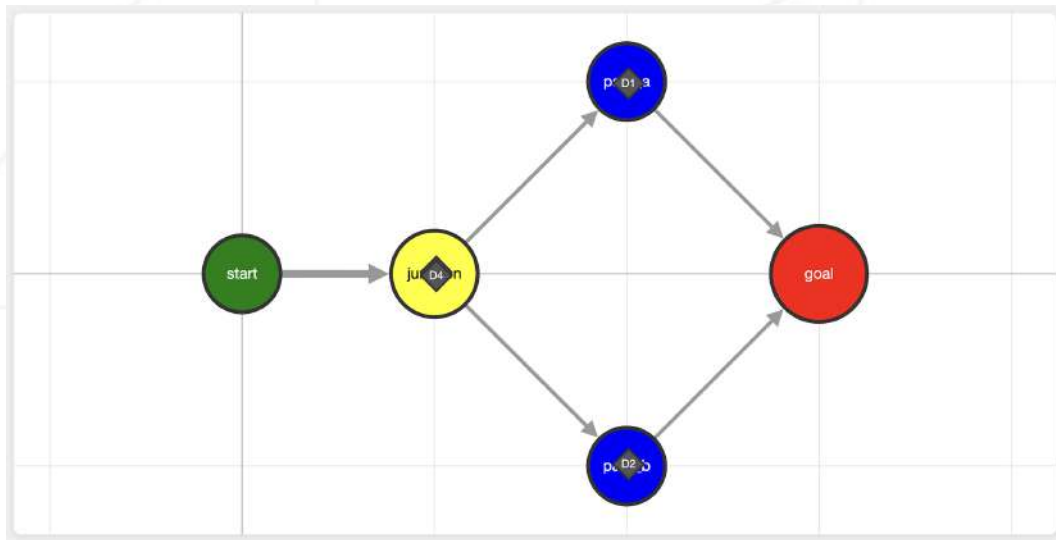


Figura VII.1: Mapa fácil 2

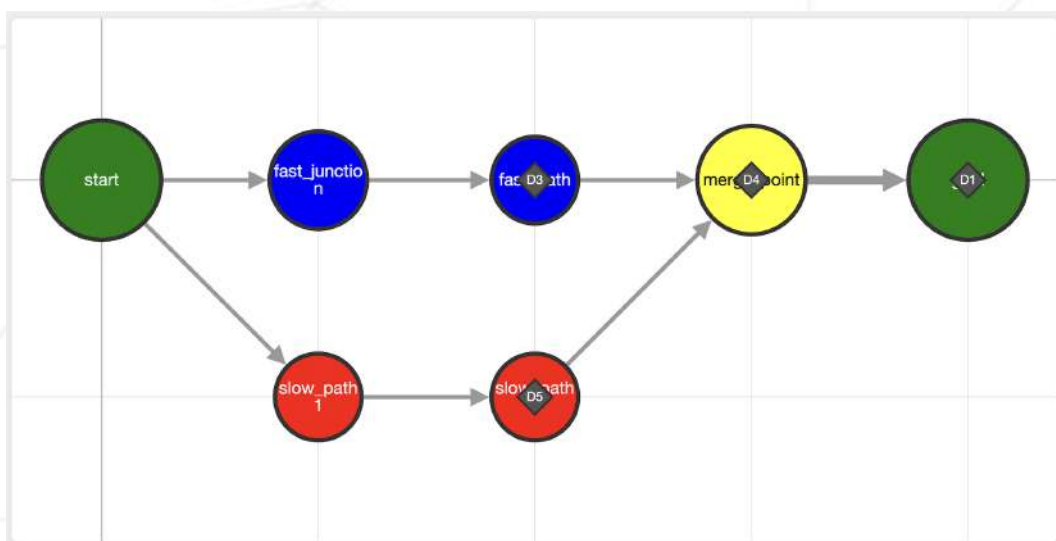
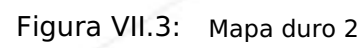


Figura VII.2: Mapa medio 3



Capítulo VIII

Requisitos de Readme

Se debe proporcionar un archivo README.md en la raíz de tu repositorio Git. Su propósito es permitir que cualquier persona no familiarizada con el proyecto (compañeros, personal, reclutadores, etc.) entienda rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El README.md debe incluir al menos:

- La primera línea debe ir en cursiva y leerse: Este proyecto ha sido creado como parte del plan de estudios 42 por <login1>[, <login2>[, <login3>[...]]].
- Una sección Descripción que presenta claramente el proyecto, incluido su objetivo y una visión general breve.
- Una sección Instrucciones que contiene información relevante sobre compilación, instalación y/o ejecución.
- Una sección Recursos que enumere referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción de cómo se usó la IA, especificando para qué tareas y qué partes del proyecto.

➡ **Podrían requerirse secciones adicionales dependiendo del proyecto (p. ej., ejemplos de uso, lista de características, opciones técnicas, etc.).**

Cualquier adición requerida se indicará explícitamente a continuación.

- También debe incluirse una descripción detallada de las elecciones de algoritmos y la estrategia de implementación.
- Documentación de las características de representación visual y de cómo mejoran la experiencia del usuario.



Tu README debe estar escrito en inglés.

Capítulo IX

Parte de bonificación

Esta parte de bonificación se revisará solo si se cumplen todos los requisitos obligatorios.

Estas son características que puedes implementar para mejorar tu proyecto:

- **Rendimiento excepcional:**
 - Tú "perfectamente" cumples los objetivos de referencia de rendimiento para todos los mapas proporcionados.
 - "perfectamente" significa que igualas o superas la cuenta de turnos objetivo.
- **Mapa desafiador:**
 - El mapa The Impossible Dream está resuelto y supera el récord de referencia de 45 turnos.

Capítulo X

Presentación y revisión por pares

Envía tu tarea en tu repositorio de Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la revisión por pares. No dudes en verificar dos veces los nombres de tus archivos para asegurarte de que son correctos.



Coloca todos tus archivos en la raíz de tu repositorio.

Una simulación totalmente funcional escrita en Python, que incluya:

- Un analizador (parser) para el formato de archivo de entrada.
- Un motor de simulación que respete las reglas de movimiento y de zonas.
- Un algoritmo de búsqueda de ruta (o múltiples) capaz de minimizar el número total de giros.
- Un sistema de representación visual (colores de terminal y/o interfaz gráfica).
- Una salida de terminal o registro que siga el formato especificado.



Tenga en cuenta que puede que le pidamos explicar su código o incluso escribir algo de código. Asegúrese de estar preparado para ello.



Los mapas de evaluación pueden ser diferentes de los proporcionados en el tema.

Durante la evaluación, ocasionalmente puede solicitarse una modificación breve del proyecto. Esto podría implicar un pequeño cambio de comportamiento, escribir unas pocas líneas de código o

reescribir, o una característica fácil de añadir.

Aunque este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso está diseñado para verificar tu comprensión real de una parte específica del proyecto. La modificación puede realizarse en cualquier entorno de desarrollo que elijas (p. ej., tu configuración habitual), y debería ser factible en unos minutos, a menos que se defina un plazo específico como parte de la evaluación.

Por ejemplo, se puede pedir que hagas una pequeña actualización a una función o script, modificar una visualización o ajustar una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.